Continuous Testing Fundamentals & Role of SDET/DevOps

Day 1 · Module 1 · 2 Hours

SDET / DevOps Engineer

Intermediate

16-Hour Programme

TOOLS

- Git
- GitHub Actions
- Jenkins
- Azure DevOps

LEARNING OUTCOMES

What You Will Achieve Today

Explain Continuous Testing in DevOps

Understand how CT fits across CI, CD & production monitoring stages.

SDET & DevOps Collaboration Models

Map the shared responsibilities between SDETs and DevOps engineers.

Identify Benefits & Challenges of CI Testing

Recognise what shifts when testing moves into the pipeline — and what breaks.

EVOLUTION

From Traditional to Continuous Testing

Traditional Testing	Continuous Testing
Testing happens at the END of development	✓ Quality checks integrated across the delivery lifecycle
Separate QA phase — often gated	✓ Automated gates reduce manual checkpoints significantly
Manual regression cycles dominate	✓ Shift-left pulls quality conversations into dev sprints
Defects found late → expensive fixes	✓ Defects caught in minutes rather than weeks
Weeks-long release cycles	✓ High-performing teams deploy frequently with safer recovery
Dev & Test teams work in silos	✓ Dev × QA × Ops share quality ownership across stages
Feedback arrives after the fact	✓ Fast feedback on commits; deeper signals on merges & deploys

STRATEGY

Shift-Left & Shift-Right: The Full Spectrum

Plan	Code	Build	Test	Release	Deploy	Operate	Monitor

SHIFT-LEFT (Prevent Defects Earlier)

SHIFT-RIGHT (Monitor & Learn in Prod)

Shift-Left Practices

- TDD/BDD — specify expected behaviour early
- Static analysis (SAST) integrated in IDE/PR
- Unit tests on every commit; contracts on PR merge
- Dev × QA pairing on acceptance criteria
- Early environment provisioning & test data prep

Shift-Right Practices

- Chaos engineering & fault injection
- Canary / blue-green deployments
- A/B testing with feature flags
- Observability, alerting & distributed tracing
- Synthetic monitoring (scripted user journeys)

Where Testing Lives in the CI/CD Pipeline

Source Control
- Pre-commit hooks
- Linting / SAST
- PR checks

→

Build & Unit Test
- mvn test / pytest
- Code coverage gates
- Fast feedback <5 min

→

Integration Test
- API contract tests
- Service-level checks
- Parallel execution

→

Staging & Regression
- UI smoke suites
- Perf baselines
- Quality gates

→

Prod Monitor
- Synthetic tests
- Alerting / Tracing
- Chaos probes

📊 Industry benchmarks (DORA research): elite teams deploy frequently, with low change failure rates and fast recovery times

◀ Continuous Feedback Loop ▶

The Test Pyramid & Strategy Alignment

E2E / UI Tests

~10% (example) of suite · Slowest

Count: Few

- Critical user journeys only
- Browser-level regression suite
- Run nightly or pre-release

Integration Tests

~30% (example) of suite · Medium

Count: Some

- API contract & schema tests
- Cross-service interactions
- Triggered on every PR merge

Unit Tests

~60% (example) of suite · Fastest

Count: Many

- Business logic & edge cases
- Fully isolated components
- Runs on every git commit

⚠ Anti-pattern: Ice Cream Cone — too many E2E, too few unit tests → slow pipelines. Percentages above are illustrative heuristics, not fixed rules.

TEAM ROLES

SDET × DevOps: Collaboration Model

SDET (Software Dev Engineer in Test)		DevOps Engineer
◈ Design & author automated test suites		◈ Design & maintain CI/CD pipeline stages
◈ Define test strategy & coverage goals		◈ Configure and provision test execution environments
◈ Code-review test code alongside developers	⟷	◈ Integrate automated test jobs into pipeline workflows
◈ Analyse test results → quality signal reporting		◈ Enforce quality gates & pipeline policy controls
◈ Champion BDD / TDD adoption in sprint planning		◈ Monitor pipeline health & delivery performance metrics
◈ Raise defects with reproduction steps & severity		◈ Coordinate release readiness & deployment approvals

Shared Ownership

Pipeline failures · Flaky test triage · Quality gate thresholds · Release readiness sign-off · Retrospectives

Key Metrics for Continuous Testing

DORA-aligned Delivery Metrics used in this course (dora.dev — historical benchmark bands, not fixed 2026 thresholds)

Deployment Frequency
Elite: Multiple/day
How often code ships to prod
DORA benchmark band

Lead Time for Changes
Elite: < 1 hour
Last commit → running in prod
DORA benchmark band

Change Failure Rate
Elite: 0–15%
Deployments needing remediation
DORA benchmark band

Failed Deploy Recovery
Elite: < 1 hour
Time to restore service
DORA benchmark band

Deployment Rework Rate
Added: 2024
Unplanned re-deploys post-incident
DORA benchmark band

Test Quality & Health Metrics (team / tool-reported — not DORA)

Test Pass Rate
Target: ≥ 95%
Suite health & build confidence
team / tooling metric

Code Coverage
Guideline: ≥ 80%
Lines/branches exercised by tests
team / tooling metric

Flaky Test Rate
Target: < 2%
Intermittent failures eroding trust
team / tooling metric

Defect Escape Rate
Target: ↓ trend
Defects reaching production
team / tooling metric

DEMO

Demo: CI/CD Pipeline with Testing Stages

01 Pipeline Overview

- Open GitHub Actions / Jenkins / Azure DevOps
- Walk through a multi-stage YAML pipeline file
- Identify: trigger, jobs, stages, steps
- Highlight: where tests are triggered (push / PR / schedule)

02 Review an Automated Test Execution in CI

- View a completed pipeline run in the UI
- Inspect the test job logs — pytest / TestNG output
- Understand pass/fail summary & exit codes
- Trace a test failure back to the failing commit

03 Identify Testing Gaps in a Pipeline

- Analyse a sample pipeline missing integration tests
- Locate the gap between unit test and deploy stage
- Discuss risk: what issues could reach production?
- Propose improvements — map to test pyramid

HANDS-ON LAB

Lab: Analyse, Map & Identify

50% Theory | 50% Hands-on

Task 1

Annotated YAML with comments

Analyse a Sample CI Pipeline

1. Fork the provided sample repository
2. Open the .github/workflows/ci.yml file
3. Identify all testing-related steps and triggers
4. Note: which test types are present and which are missing

Task 2

Completed mapping matrix

Map Test Types to Pipeline Stages

1. Create a mapping table (Stage → Test Type → Tool)
2. Use the CI stages from Task 1 as rows
3. Classify: Unit / Integration / Contract / E2E / Perf
4. Align with the test pyramid ratios discussed

Task 3

Automation opportunity checklist

Identify Automation Opportunities

1. Review the mapping gaps from Task 2
2. Prioritise: what should be automated first?
3. Estimate effort level: Low / Medium / High
4. Document findings in the strategy worksheet

Day 1 Deliverables & Key Takeaways

Deliverables You Take Away

- Continuous Testing Strategy Worksheet
- CI Test Coverage Mapping Document
- Annotated Pipeline YAML with test gaps noted

Key Takeaways

1. Continuous testing means quality signals at every stage — not just a final gate
2. Shift-left catches defects early; shift-right observes and validates production behaviour
3. The test pyramid guides investment: maximise unit tests, minimise E2E
4. SDET and DevOps engineers co-own pipeline quality and release readiness
5. DORA metrics help measure software delivery throughput and stability across the pipeline

Tomorrow — Day 2: Test Automation Frameworks for CI/CD · Framework design, TestNG/JUnit/PyTest, Maven/Gradle integration

Hands-on: Set up automation framework + CLI execution + report generation

RECAP QUIZ

Module 1 – Quick Knowledge Check

Q1. What is the primary goal of shift-left testing?

- Detect and fix defects earlier in the SDLC to reduce rework cost and speed up delivery.

Q2. Name two shift-right testing techniques.

- Any two: chaos engineering, canary releases, A/B testing with feature flags, synthetic monitoring, production observability.

Q3. In the test pyramid, which layer should have the highest count?

- Unit tests — fast, isolated, cheapest to run; ~60% of the overall test suite.

Q4. What does DORA's Change Failure Rate measure?

- The share of deployments that cause a production failure and require remediation. Lower is better; DORA tracks it as a stability indicator across performance bands.